Parametric Polymorphism in C++, Java, and C#: Generics vs. Templates and Metaprogramming Power

Elia Leonardi
University of Pisa

July 2026

Contents

Chapter 1

AI Prompt Engineering

For the technical accuracy of the output, following the guidelines of the project, the report is generated using a **Two-phase prompt strategy**: prompt improvement and prompt execution.

1.1 Prompt Engineering Phases

1.1.1 Prompt Improvement

The first phase is prompt improvement, where the initial prompt is redefined and improved following the project guidelines and a rigorous academic approach. Every prompt improvement is independent from the previous prompt, and the output is generated using the improved prompt.

1.1.2 Prompt Execution

The second phase is prompt execution, where the improved prompt is used to generate the final output. In this phase, the system is invited to also give the source references used to generate the output.

Chapter 2

Prompts

2.1 Prompt 0: Configure Prompt Engineering

Model: Gemini 3.1 Pro extended reasoning

Prompt:

System Persona
Act as a world-class computer scientist and principal compiler engineer specializing in Programming Language Theory (PLT), virtual machine architectures (such as the .NET CLR and JVM), and type systems.

Goal
Generate an exhaustive, publication-grade academic report section in LaTeX titled "Instantiation Time in C#". The text must analyze the instantiation mechanics and timeline of C# generics during runtime execution and rigorously contrast this specific temporal approach with the compile-time instantiation model of C++ templates and the type erasure model of Java generics.

Requirements

- Focus explicitly on the instantiation timeline and execution model of C# generics, detailing when the Common Language Runtime (CLR) and the Just-In-Time (JIT) compiler process generic types.
- Explain the concept of reified generics in C# strictly in terms of how type information is preserved for the runtime instantiation phase.
- Detail the dual-phase implementation mechanisms used by C# (compile-time CIL generation vs. runtime JIT native code generation), including reference-type sharing and value-type specialization, strictly from an instantiation perspective.
- Provide a rigorous technical comparison of the timeline between C# (JIT-time specialization) and C++ (ahead-of-time compile-time monomorphization).
- Provide a rigorous technical comparison of the timeline between C# (runtime instantiation) and Java (compile-time type erasure with no runtime instantiation).

- STRICT SCOPE CONTROL: You must confine the generated text strictly to the instantiation timeline, mechanisms, and architectural flow. You must actively avoid tangential subjects. DO NOT discuss executable size, code bloat, memory footprint, garbage collection overhead, runtime performance speed, or dynamic dispatch overhead.
- Base every technical statement on authoritative references (e.g., C# Language Specification, ECMA-335, JVM Specification).
- Require consistent citation formatting and a final bibliography section.
- Output complete, compilation-ready LaTeX code.

Writing Style Constraint

- Produce a complete technical-academic text suitable for integration into a larger research document rather than a short summary.
- Maintain a formal, objective, and precise scientific writing style comparable to ACM or IEEE publications.
- Use accurate compiler and virtual machine terminology.
- Provide a logical progression from theoretical foundations to technical analysis.
- Avoid first-person pronouns and conversational language.
- Output ONLY valid LaTeX code. Do not include markdown formatting outside of the LaTeX structure, and do not add conversational introductory or concluding remarks.

Output Structure
Organize the report section according to the following hierarchy:

1. Instantiation Time in C#
2. CLR and JIT Generic Instantiation Mechanism
3. Comparison with C++ Templates: Runtime Instantiation vs. Compile-Time Instantiation
4. Comparison with Java Generics: Runtime Instantiation vs. Type Erasure
5. References

Output:
Add to the context of the model the persona, formatting constraints and the goal of the report.

2.2 Prompt 1: Overview of the Parametric Polymorphism

Model: Gemini 3.1 Pro extended reasoning.

Goal: Generate an overview of the Parametric Polymorphism and ad-hoc polymorphism. Giving details of type erasure, compared with generics, and monomorphization. Also compared differences between source code, byte code and binary code.

Prompt:

- Provide an exhaustive theoretical background on parametric polymorphism.
- Define and thoroughly explain Christopher Strachey's distinction between parametric polymorphism and ad-hoc polymorphism.
- Detail the two primary implementation strategies discussed in the literature:
    1. Homogeneous translation (type erasure), detailing how it results in one compiled representation for all instantiations.
    2. Heterogeneous translation (reification/monomorphization), detailing how it results in separate code generated per instantiated type.
- Analyze type erasure versus reified generics as the core axis of comparison.
- Analyze compile-time versus runtime characteristics as the second axis of comparison, specifically examining when instantiation happens and what type information survives into the compiled binary or bytecode.
- The entire document must be written in professional, compilation-ready LaTeX code.

Resources:
The generation of the technical report was supported by the following academic and technical references. The selected resources provide the theoretical foundations of parametric polymorphism, the classification of polymorphic systems, and the implementation techniques adopted by modern programming languages.

The theoretical background was derived from foundational works on type systems and polymorphism, including Strachey's original classification of polymorphism, Cardelli and Wegner's analysis of type abstraction and polymorphism, and Pierce's formal treatment of parametric polymorphism and type systems [?, ?, ?, ?].

The analysis of generic implementation strategies, including homogeneous translation, type erasure, heterogeneous translation, reification, and monomorphization, was based on research concerning generic programming implementations and runtime systems, including the .NET Common Language Runtime and Java generic mechanisms [?, ?, ?, ?, ?, ?, ?].

The comparison between compile-time and runtime characteristics, including instantiation time, runtime representation, reflection capabilities, runtime type information, optimization opportunities, performance trade-offs, and type safety guarantees, was supported by compiler literature and modern programming language implementation documentation [?, ?, ?, ?, ?, ?, ?].

2.3 Prompt 2: Instantiation Time in C++ Templates

Goal: Generate a detailed and technically rigorous section on the *instantiation time* of C++ templates, with an exclusive focus on compile-time instantiation mechanisms and their implications for compiler execution.

Context:
This prompt extends the previous output generated for the *Instantiation Time* section in Prompt 1 (Section 2.1). The objective is to deepen the discussion by analyzing the complete compile-time instantiation model adopted by C++ templates from both theoretical and compiler implementation perspectives.

Prompt:
Act as a world-class computer scientist and principal compiler engineer specializing in Programming Language Theory (PLT), type systems, and C++ compiler execution models.

Generate an exhaustive, publication-grade academic report focused exclusively on the *instantiation time* of parametric polymorphism in C++ templates.

The report must satisfy the following requirements.

Requirements

- Focus exclusively on the compile-time instantiation model adopted by C++ templates. Any discussion of Java or C# generics should be excluded.
- Explain the complete compile-time instantiation pipeline, including:
  - template argument deduction;
  - two-phase name lookup;
  - implicit and explicit template instantiation;
  - monomorphization and specialization.
- Describe how concrete template uses trigger the generation of specialized functions and classes during compilation, emphasizing that no template instantiation occurs during program execution.
- Explain the compilation and linking process, discussing how multiple translation units may independently instantiate identical template specializations and how modern linkers eliminate duplicates using mechanisms such as COMDAT sections, weak symbols, or equivalent implementation techniques.

- Analyze the consequences of compile-time instantiation with respect to:
  - compilation time;
  - executable size (code bloat);
  - optimization opportunities;
  - runtime performance.
- Whenever appropriate, relate the implementation strategy to the theoretical notion of type substitution in parametric polymorphism.
- Base every technical statement on authoritative references, prioritizing the ISO/IEC 14882:2020 C++ Standard and established compiler literature.
- Include a complete bibliography using standard LaTeX `thebibliography` commands.

Writing Style

- Adopt a formal, objective, and precise scientific writing style comparable to ACM or IEEE publications.
- Use accurate compiler terminology (e.g., *template argument deduction*, *two-phase lookup*, *monomorphization*, *translation unit*, *COMDAT*, *weak symbols*, *link-time deduplication*).
- Avoid first-person pronouns and conversational language.
- Provide technically detailed explanations rather than high-level summaries.

Output Structure
The generated document must consist entirely of valid, self-contained, compilation-ready LaTeX code. Organize the report according to the following hierarchy:

1. Introduction: The C++ Template Instantiation Paradigm
2. The Compile-Time Instantiation Pipeline
3. Monomorphization and Specialization
4. Link-Time Deduplication (COMDAT and Weak Symbols)
5. Trade-offs: Compile-Time Overhead vs. Runtime Performance
6. Conclusion

Primary References

- **ISO/IEC JTC 1/SC 22/WG 21, *ISO/IEC 14882:2020 Programming Languages — C++*.** The normative reference for C++ template semantics, including template declarations, template argument deduction, implicit and explicit template instantiation, specialization mechanisms, two-phase name lookup, and the rules governing compile-time template processing [**?**].

- **David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017.** The primary implementation-oriented reference for C++ template instantiation, covering implicit instantiation, explicit instantiation, template specialization, instantiation models, compilation dependencies, and the impact of template expansion on generated code size and compilation time [?].

- **Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley Professional, 2013.** Provides the language-level description of C++ templates, including generic programming, compile-time polymorphism, template specialization, and the role of templates in static code generation [?].

- **Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools*, 3rd Edition, Pearson, 2022.** Provides the general compiler theory background required to analyze template instantiation, including translation phases, intermediate representations, code generation, optimization, separate compilation, and the relationship between source-level abstractions and generated machine code [?].

- **Keith D. Cooper and Linda Torczon, *Engineering a Compiler*, 3rd Edition, Morgan Kaufmann, 2022.** Supports the compiler-engineering analysis of specialization strategies, compile-time transformations, optimization opportunities, and the trade-offs between generated-code specialization and compilation overhead [?].

- **Benjamin C. Pierce, *Types and Programming Languages*, MIT Press, 2002.** Provides the theoretical foundation for parametric polymorphism, universal quantification, type abstraction, and type substitution, which form the theoretical basis for generic programming mechanisms such as C++ templates [?].

- **Jean-Yves Girard, "Interprétation fonctionnelle et élimination des coupures de l'arithmétique d'ordre supérieur," 1972, and John C. Reynolds, "Towards a Theory of Type Structure," 1974.** These works introduce the formal foundations of System F and universal quantification, providing the type-theoretic interpretation of parametric polymorphism and type application mechanisms [?, ?].

- **Luca Cardelli and Peter Wegner, "On Understanding Types, Data Abstraction, and Polymorphism," ACM Computing Surveys, 1985.**

2.4 Prompt 3: Instantiation Time in Java Generics

Goal: Generate a detailed and technically rigorous section on the *instantiation time* of Java Generics, with an exclusive focus on compile-time instantiation mechanisms and their implications for compiler execution.

Context:
This prompt extends the previous output generated for the *Instantiation Time* section in Prompt 1 (Section 2.1). The objective is to deepen the discussion by analyzing the complete compile-time instantiation model adopted by C++ templates from both theoretical and compiler implementation perspectives.

Prompt:
The following requirements extend the previous analysis and refine the discussion of the *instantiation time* dimension of parametric polymorphism. The focus must remain exclusively on the Java Generics implementation model, with C++ templates considered only as a comparative reference.

Additional Requirements

- Provide a detailed analysis of the Java Generics compilation model, explaining how generic declarations are processed by the Java compiler during compilation and how generic type information is transformed into JVM-compatible representations.
- Describe the complete Java Generics compilation pipeline, including:
  - compile-time verification of generic constraints and type bounds;
  - type checking of parameterized types and generic method invocations;
  - type erasure transformation;
  - replacement of type variables with their erasures;
  - generation of synthetic bridge methods required to preserve polymorphic dispatch after erasure;
  - insertion of implicit runtime casts in the generated bytecode.
- Explain how Java generic types are reduced to erased JVM-level representations, emphasizing that parameterized type arguments are not preserved as runtime class identities.
- Analyze the consequences of the Java type erasure model with respect to:
  - compilation time and compiler workload;
  - bytecode footprint and class-file size;
  - absence of specialized generic implementations;
  - runtime execution overhead;
  - primitive boxing and unboxing;
  - memory indirection and cache locality effects.
- Compare the Java Generics implementation strategy with the C++ template instantiation model only to highlight the difference between:
  - Java erasure-based translation, which produces a single erased representation compatible with the JVM;

  - C++ compile-time template instantiation, which generates specialized implementations through monomorphization.

- Discuss the semantic consequences of Java's lack of runtime generic type information, including:
  - non-reifiable generic types;
  - limitations of runtime type checks involving parameterized types;
  - restrictions on generic array creation;
  - reflection limitations caused by type erasure.

Primary References

- **Oracle, *The Java Language Specification, Java SE 24 Edition.*** The primary normative reference for Java Generics semantics. It should be cited for generic declarations, type variables, parameterized types, bounded type parameters, compile-time type checking, type erasure rules, reifiable types, restrictions on `instanceof`, generic array creation, and unchecked conversions [**?**].

- **Oracle, *The Java Virtual Machine Specification, Java SE 24 Edition.*** The primary normative reference for the JVM execution model. It should support claims concerning class-file representation, method descriptors, bytecode generation after erasure, runtime type checking, and the absence of parameterized generic type information in the JVM runtime representation [**?**].

- **Gilad Bracha, Martin Odersky, David Stoutamire, and Philip Wadler, "Making the Future Safe for the Past: Adding Genericity to the Java Programming Language," OOPSLA 1998.** The foundational reference for the design of Java Generics. It supports the discussion of type erasure as a compilation strategy, backward compatibility with existing JVM binaries, and the translation of parameterized types into non-generic bytecode representations [**?**].

- **Gilad Bracha, *Generics in the Java Programming Language*, Sun Microsystems, 2004.** This reference supports implementation-level discussions of Java Generics, including compile-time generic checking, erasure translation, bridge method generation, and compatibility between generic source code and the JVM execution model [**?**].

- **Atsushi Igarashi, Benjamin C. Pierce, and Philip Wadler, "Featherweight Java: A Minimal Core Calculus for Java and GJ," TOPLAS 2001.** This work provides a formal foundation for the analysis of Java's generic type system, including type substitution, inheritance, generic classes, and the translation of Generic Java programs [**?**].

- **Benjamin C. Pierce, *Types and Programming Languages*, MIT Press, 2002.** This reference provides the theoretical foundation for parametric polymorphism, universal quantification, type abstraction, and substitution-based models of generic programming languages [**?**].

- **Luca Cardelli and Peter Wegner, "On Understanding Types, Data Abstraction, and Polymorphism," ACM Computing Surveys, 1985.** This reference supports the classification of polymorphism and the theoretical distinction between parametric polymorphism, subtype polymorphism, and ad-hoc polymorphism [**?**].

- **Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley Professional, 2013.** This reference supports the comparison with C++ templates, particularly compile-time template instantiation, specialization, and static polymorphism [**?**].

- **David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017.** This reference supports compiler-level aspects of C++ template instantiation, including implicit instantiation, explicit specialization, template expansion, and the effects of monomorphization on generated code size [**?**].

- **ISO/IEC JTC 1/SC 22/WG 21, *ISO/IEC 14882:2024 Programming Languages — C++*.** The normative reference for C++ template semantics, including template argument deduction, instantiation rules, specialization mechanisms, and compile-time template processing [**?**].

- **Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman, *Compilers: Principles, Techniques, and Tools*, 3rd Edition, Pearson, 2022.** This reference supports general compiler implementation concepts, including translation phases, intermediate representations, optimization, code generation, and the relationship between source-level abstractions and executable representations [**?**].

- **Keith D. Cooper and Linda Torczon, *Engineering a Compiler*, 3rd Edition, Morgan Kaufmann, 2022.** This reference supports compiler engineering aspects related to translation strategies, specialization, optimization, and generated-code management [**?**].

2.5 Prompt 4: Instantiation Time in C# Generics

Goal: Generate a detailed and technically rigorous section on the *instantiation time* of C# generics, with an exclusive focus on runtime instantiation mechanisms and their implications for compiler execution.

Context:
This prompt extends the previous output generated for the *Instantiation Time* sec-

tion in Prompt 1 (Section 2.1). The objective is to deepen the discussion by analyzing the complete runtime instantiation model adopted by C# generics from both theoretical and compiler implementation perspectives.

Prompt:

System Persona
Act as a world-class computer scientist and principal compiler engineer specializing in Programming Language Theory (PLT), virtual machine architectures (such as the .NET CLR and JVM), and type systems.

Goal
Generate an exhaustive, publication-grade academic report section in LaTeX titled "Instantiation Time in C#". The text must analyze the instantiation mechanics and timeline of C# generics during runtime execution and rigorously contrast this specific temporal approach with the compile-time instantiation model of C++ templates and the type erasure model of Java generics.

Requirements

- Focus explicitly on the instantiation timeline and execution model of C# generics, detailing when the Common Language Runtime (CLR) and the Just-In-Time (JIT) compiler process generic types.
- Explain the concept of reified generics in C# strictly in terms of how type information is preserved for the runtime instantiation phase.
- Detail the dual-phase implementation mechanisms used by C# (compile-time CIL generation vs. runtime JIT native code generation), including reference-type sharing and value-type specialization, strictly from an instantiation perspective.
- Provide a rigorous technical comparison of the timeline between C# (JIT-time specialization) and C++ (ahead-of-time compile-time monomorphization).
- Provide a rigorous technical comparison of the timeline between C# (runtime instantiation) and Java (compile-time type erasure with no runtime instantiation).
- STRICT SCOPE CONTROL: You must confine the generated text strictly to the instantiation timeline, mechanisms, and architectural flow. You must actively avoid tangential subjects. DO NOT discuss executable size, code bloat, memory footprint, garbage collection overhead, runtime performance speed, or dynamic dispatch overhead.
- Base every technical statement on authoritative references (e.g., C# Language Specification, ECMA-335, JVM Specification).
- Require consistent citation formatting and a final bibliography section.
- Output complete, compilation-ready LaTeX code.

Writing Style Constraint

- Produce a complete technical-academic text suitable for integration into a larger research document rather than a short summary.
- Maintain a formal, objective, and precise scientific writing style comparable to ACM or IEEE publications.
- Use accurate compiler and virtual machine terminology.
- Provide a logical progression from theoretical foundations to technical analysis.
- Avoid first-person pronouns and conversational language.
- Output ONLY valid LaTeX code. Do not include markdown formatting outside of the LaTeX structure, and do not add conversational introductory or concluding remarks.

Output Structure
Organize the report section according to the following hierarchy:

1. Instantiation Time in C#
2. CLR and JIT Generic Instantiation Mechanism
3. Comparison with C++ Templates: Runtime Instantiation vs. Compile-Time Instantiation
4. Comparison with Java Generics: Runtime Instantiation vs. Type Erasure
5. References

Primary References

- **Ecma International, *ECMA-334: C# Language Specification*, 2023.** This reference provides the normative specification of the C# language, including the definition of generic types, generic methods, type parameters, constraints, and the language-level semantics of generic programming in C# [**?**].
- **Ecma International, *ECMA-335: Common Language Infrastructure (CLI)*, 2024.** This reference defines the runtime model underlying C# execution, including the representation of generic types in metadata, Common Intermediate Language (CIL), runtime type construction, and the mechanisms used by the CLR to execute generic programs [**?**].
- **Andrew Kennedy and Don Syme, *Design and Implementation of Generics for the .NET Common Language Runtime*, Proceedings of PLDI, ACM, 2001.** This reference provides the main technical foundation for the implementation of generics in the .NET Common Language Runtime. It describes the design choices behind runtime generic instantiation, the interaction between generic metadata and JIT compilation, and the different representation strategies used for reference types and value types [**?**].

- **Jeffrey Richter, *CLR via C#*, 4th Edition, Microsoft Press, 2012.** This reference explains the internal execution model of the Common Language Runtime, including assembly loading, metadata resolution, Just-In-Time (JIT) compilation, runtime type representation, and the practical behavior of C# generics during program execution [**?**].
- **Gilad Bracha, *Generics in the Java Programming Language*, Sun Microsystems, 2004.** This reference describes the design and implementation principles of Java generics, including compile-time type checking, type erasure, and the absence of runtime generic specialization in the Java Virtual Machine [**?**].
- **Oracle, *The Java Language Specification, Java SE 24 Edition*, 2024.** This reference provides the formal specification of Java generics, including generic declarations, type parameter handling, parameterized types, and the compile-time rules that define the behavior of generic programs before execution [**?**].
- **Oracle, *The Java Virtual Machine Specification, Java SE 24 Edition*, 2024.** This reference describes the runtime execution model of the Java Virtual Machine, including bytecode representation, runtime type information, and the consequences of type erasure for generic types during execution [**?**].
- **David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017.** This reference supports the comparison with C++ templates by describing compile-time template instantiation, specialization mechanisms, and the generation of concrete implementations through compiler-based monomorphization [**?**].

2.6 Prompt 4: Runtime representation of C++ templates

goal: Generate a detailed and technically rigorous section on the *runtime representation* of C++ templates, with an exclusive focus on compile-time instantiation mechanisms and their implications for runtime execution. **context**: This prompt extends the previous output generated for the *Instantiation Time* section in Prompt 1 (Section 2.1). **prompt**: **System Persona**
Act as a world-class computer scientist and principal compiler engineer specializing in Programming Language Theory (PLT), compiler construction (specifically GCC and Clang backends), and C++ architecture.

Goal
Generate an exhaustive, publication-grade academic report section in LaTeX titled "Runtime Representation of C++ Templates". The text must analyze exactly how instantiated C++ templates exist during program execution and detail the architectural consequences of compile-time monomorphization on the final executable binary.

Requirements

- Focus explicitly on the runtime representation of C++ templates. Detail what happens *after* compilation and linking, when the binary is loaded into memory.
- Explain the concept of heterogeneous translation (monomorphization) strictly in terms of how it creates distinct, independent native machine code artifacts for each type argument.
- Clearly explain the absence of generic metadata at runtime. Clarify that the execution environment (the CPU/OS) has no concept of the original "template"; it only sees ordinary functions and objects.
- Briefly address the limited type information that survives (e.g., name mangling and RTTI - Run-Time Type Information) and how it differs from reified generics.
- STRICT SCOPE CONTROL: You must confine the generated text strictly to the physical runtime representation and machine code generation. You must actively avoid tangential subjects. DO NOT discuss compilation times, macro-level code bloat metrics, manual memory management, or CPU cache performance. DO NOT compare with Java or C#; focus exclusively on the C++ architectural model.
- Base every technical statement on authoritative references (e.g., ISO C++ Standard, "C++ Templates: The Complete Guide" by Vandevoorde/Josuttis).
- Require consistent citation formatting and a final bibliography section.
- Output complete, compilation-ready LaTeX code.

Writing Style Constraint

- Produce a complete technical-academic text suitable for integration into a larger research document.
- Maintain a formal, objective, and precise scientific writing style comparable to ACM or IEEE publications.
- Use accurate compiler terminology (e.g., monomorphization, translation unit, name mangling, native machine code, RTTI).
- Provide a logical progression from theoretical mechanisms to the final binary artifact.
- Avoid first-person pronouns and conversational language.
- Output ONLY valid LaTeX code. Do not include markdown formatting outside of the LaTeX structure, and do not add conversational introductory or concluding remarks.

Output Structure
Organize the report section according to the following hierarchy:

1. Runtime Representation of C++ Templates
2. Monomorphization and Native Code Artifacts
3. Absence of Generic Metadata and the Role of RTTI
4. References

Primary references

- **David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017.** This reference provides the main technical foundation for C++ template implementation, including template instantiation, specialization mechanisms, and the translation process that transforms generic definitions into concrete compiled entities [**?**].
- **ISO/IEC JTC 1/SC 22/WG 21, *ISO/IEC 14882:2024 Programming Languages — C++*.** The normative specification for the C++ language. It defines the formal semantics of templates, including template declarations, implicit and explicit instantiation, specialization rules, and compile-time processing of template entities [**?**].
- **Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley Professional, 2013.** Provides the general C++ language model and explains the relationship between templates, type abstraction, specialization, and the generation of concrete program entities during translation [**?**].
- **Stanley B. Lippman, *Inside the C++ Object Model*, Addison-Wesley Professional, 1996.** This reference supports the analysis of C++ runtime representation, describing compiler-generated structures, object representation, and implementation-level details of how C++ abstractions are mapped into executable artifacts [**?**].
- **The Itanium C++ ABI Project, *Itanium C++ ABI: C++ ABI Specification*.** This reference documents binary-level implementation details used by C++ compiler toolchains, including symbol representation, name mangling, Run-Time Type Information (RTTI), and conventions for representing C++ entities inside compiled binaries [**?**].
- **GNU Project, *G++ Implementation Details*.** This reference describes implementation aspects of the GCC C++ compiler, including compiler-specific mechanisms for handling templates, instantiation, and generation of concrete compiled entities [**?**].

- **LLVM Project, *Clang Internals Manual.*** This reference provides information about the internal architecture of the Clang compiler, including C++ semantic analysis, template processing, and the transformation of source-level constructs into compiler intermediate representations [**?**].

- **LLVM Project, *LLVM Language Reference Manual.*** This reference describes the LLVM intermediate representation and the compiler backend model used to transform high-level language constructs into lower-level representations suitable for native machine code generation [**?**].

- **Margaret A. Ellis and Bjarne Stroustrup, *The Annotated C++ Reference Manual*, Addison-Wesley Professional, 1990.** This reference provides historical and technical documentation of the C++ language model, including early formal descriptions of templates and their translation semantics [**?**].

2.7 prompt 5: Runtime Representation of java Generics

goal: Generate a detailed academic section of runtime representation of java generics, with explicative example and comparison with C++ templates. The focus must be on the runtime representation of java generics. **prompt**: Act as a world-class computer scientist and principal compiler engineer specializing in Programming Language Theory (PLT), type systems, and the runtime execution models of both the Java Virtual Machine (JVM) and C++ compiled binaries.

Goal

Generate an exhaustive, publication-grade academic report focused exclusively on comparing the runtime representation of parametric polymorphism, specifically analyzing Java Generics and C++ Templates.

Requirements

- Analyze the runtime representation of Java Generics, thoroughly explaining the mechanism of type erasure. Discuss how the JVM handles generic types at runtime, including the generation of bridge methods, implicit type cast insertions, and the implications for primitive types and runtime type information (RTTI).

- Analyze the runtime representation of C++ Templates, detailing the monomorphization (reification) process. Explain how the C++ compiler generates distinct, specialized machine code for each instantiated type and how this manifests in the final executable memory space.

- Provide a rigorous technical comparison between the two execution models, explicitly addressing the trade-offs regarding runtime execution perfor-

mance, memory footprint (e.g., instruction cache bloat in C++ versus boxing/unboxing overhead in Java), and the preservation of type metadata at runtime.

- Distinguish clearly between the theoretical concepts of type substitution and the concrete implementation mechanisms enforced by the respective language runtimes.
- Base every technical statement on authoritative references, prioritizing the Java Language Specification (JLS), the Java Virtual Machine Specification (JVMS), the ISO/IEC 14882 C++ Standard, and established compiler literature.
- **ENFORCE STRICT SCOPE CONTROL:** Confine the generated text strictly to the requested core topic of runtime representations and their direct architectural consequences. Actively avoid tangential subjects such as syntax tutorials, compilation time comparisons, garbage collection overhead, or broader language feature comparisons not explicitly tied to runtime representation.

Writing Style Constraint

- Produce a complete technical-academic report rather than a short summary.
- Maintain a formal, objective, and precise scientific writing style comparable to ACM or IEEE peer-reviewed publications.
- Use accurate computer science and compiler terminology (e.g., *type erasure*, *monomorphization*, *reification*, *type bound*, *bridge method*, *translation unit*).
- Avoid first-person pronouns and conversational language entirely.
- Provide a logical progression from theoretical foundations to technical analysis, ensuring detailed explanations over high-level summaries.
- Avoid unsupported claims and clearly identify any architectural assumptions.

Output Structure

The generated document must consist entirely of valid, self-contained, compilation-ready LaTeX code. Ensure compatibility with standard LaTeX compilation workflows (e.g., pdfLaTeX). Preserve a professional academic document structure, including appropriate sections, mathematical notation, tables, figures, citations, and bibliography commands. Include a complete bibliography using standard LaTeX `thebibliography` commands to document all authoritative academic and technical sources. Organize the report according to the following hierarchy:

1. Introduction: Parametric Polymorphism at Runtime
2. Java Generics: The Type Erasure Model

3. C++ Templates: The Monomorphization Model
4. Comparative Analysis: Memory Footprint, Execution Speed, and Type Metadata
5. Conclusion

Primary references

- **James Gosling, Bill Joy, Guy Steele, Gilad Bracha, and Alex Buckley, *The Java Language Specification*, Java SE 21 Edition, Oracle, 2023.** The normative specification of the Java language. Chapter 4 formally defines the type system and the type erasure transformation used to implement Java Generics, including erasure of type variables, bounds, raw types, and generic subtyping rules. This is the primary reference for the semantics of Java Generics and their translation before execution [**?**].
- **Tim Lindholm, Frank Yellin, Gilad Bracha, Alex Buckley, Daniel Smith, and Gavin Bierman, *The Java Virtual Machine Specification*, Java SE 21 Edition, Oracle, 2023.** The normative specification of the Java Virtual Machine. It describes the JVM class file format, method descriptors, bytecode instructions such as `checkcast`, the `Signature` attribute used to preserve generic metadata, and the runtime execution model on which erased generics operate [**?**].
- **Gilad Bracha, Martin Odersky, David Stoutamire, and Philip Wadler, "Making the Future Safe for the Past: Adding Genericity to the Java Programming Language," *Proceedings of OOPSLA*, ACM, 1998.** The original paper introducing Java Generics. It motivates the adoption of type erasure, discusses backward compatibility with existing JVM binaries, and explains the architectural reasons for choosing erasure instead of runtime reification [**?**].
- **Gilad Bracha, *Generics in the Java Programming Language*, Sun Microsystems, 2004.** Provides a detailed explanation of Java Generics, including the implementation of type erasure, bridge methods, wildcard types, compiler-generated casts, and their interaction with the JVM execution model [**?**].
- **Joshua Bloch, *Effective Java*, 3rd Edition, Addison-Wesley Professional, 2018.** Discusses the practical implications of Java's erased generic representation, including type safety, bridge methods, raw types, boxing and unboxing, and the limitations imposed by the lack of reified generic types [**?**].
- **Brian Goetz, *Java Concurrency in Practice*, Addison-Wesley Professional, 2006.** Although primarily focused on concurrency, this reference discusses generic type safety, erased runtime representations, and the interaction between generic types and the Java memory model [**?**].

- **Bill Venners, *Inside the Java Virtual Machine*, 2nd Edition, McGraw-Hill, 2000.** Provides an implementation-oriented description of the JVM architecture, runtime data areas, bytecode execution, dynamic linking, and runtime type information, offering useful background for understanding how erased generic types execute on the JVM [?].

- **Richard Warburton, *The Java 8 Language Specification: Generics and Type Inference*, Oracle Technical Documentation.** Provides additional discussion of generic type inference, erased type representations, compiler-generated casts, and bridge methods introduced during compilation [?].

2.8 Prompt 6: Runtime Representation of C# Generics

goal: Generate a detailed academic section of runtime representation of C# generics with explicit example and comparison with C++ template and Java generics.
prompt: **System Persona**
You are a Principal Software Engineer and Researcher specializing in programming language theory, virtual machine implementation, and compiler design. You possess deep academic and industry knowledge of the .NET Common Language Runtime (CLR), the Java Virtual Machine (JVM), and C++ compilation pipelines.

Goal
Write a rigorous, publication-quality technical-academic report section analyzing the runtime representation of C# generics. Your analysis must contrast the C# implementation with the runtime representations of C++ templates and Java generics to highlight the structural trade-offs, architectural differences, and design philosophies inherent to each approach.

Requirements

- **Core Topic Analysis:** Detail the runtime representation of C# generics, specifically focusing on how the .NET Common Language Runtime (CLR) handles reification. Explain how the compiler emits Common Intermediate Language (CIL) metadata and how the Just-In-Time (JIT) compiler dynamically instantiates generic classes and methods at runtime.

- **Mechanisms of C# Reification:** Analyze the differing runtime behaviors of C# generics when instantiated with value types (which generate specialized machine code per type) versus reference types (which share a single native implementation using pointer representations, mitigated by runtime dictionary passing or execution-engine tables).

- **Contrast with Java Generics (Type Erasure):** Contrast C# runtime reification with Java's runtime representation. Explain how Java's compile-

time Type Erasure removes generic type arguments, resulting in a shared runtime representation with no generic metadata, casting overhead, and the inability to instantiate generic types directly.

- **Contrast with C++ Templates (Monomorphization):** Contrast C# runtime reification with C++ template instantiation. Explain how C++ utilizes a heterogeneous translation model where templates are fully resolved and expanded at compile-time into distinct machine code artifacts. Highlight that C++ executables lack formal runtime template representation, metadata, or dynamic generic instantiation capabilities.
- **Theoretical vs. Implementation Distinction:** Clearly distinguish between the theoretical paradigms of generics and their underlying machine/virtual-machine implementations (e.g., virtual method tables, metadata, dictionary passing, memory layouts).
- **Academic References:** Rely on authoritative sources (e.g., official language specifications like ECMA-335, peer-reviewed publications). Use consistent citation formatting and include a final bibliography section.

Writing Style Constraint

- Maintain a formal, objective, and authoritative academic tone.
- Produce a complete technical-academic report, providing a logical progression from foundations to analysis.
- Use precise technical terminology (e.g., *parametric polymorphism*, *reification*, *type erasure*, *monomorphization*, *JIT specialization*).
- Avoid colloquial language or unsupported generalizations.
- Format technical variables or formal representations using LaTeX notation (e.g., $T\langle U\rangle$).
- **STRICT SCOPE CONTROL:** Confine the analysis strictly to the runtime representation, metadata structures, compilation pipelines, and execution engine mechanics. Actively avoid tangential subjects such as application benchmarks, hardware performance, unrelated garbage collection, or general syntax ergonomics.
- Begin directly with the technical content. Do not include conversational filler.

Output Structure

Structure the report logically using the following exact headers:

1. Introduction to C# Generic Reification

2. CLR Execution Engine and Runtime Representation
3. Comparative Runtime Paradigm: C# vs. Java Generics
4. Comparative Runtime Paradigm: C# vs. C++ Templates
5. Architectural Synthesis
6. References

Under "Architectural Synthesis", include a concise comparison table summarizing the features, metadata presence, compilation model, and representation strategy across C#, Java, and C++.
primary references **Primary references**

- **ECMA International, *ECMA-335: Common Language Infrastructure (CLI).*** Used as the primary reference for the CLR execution model, CIL metadata structures, generic parameter representation, and runtime reification mechanisms.
- **Andrew Kennedy and Don Syme, *Design and Implementation of Generics for the .NET Common Language Runtime.*** Used for the analysis of CLR generic implementation strategies, including JIT specialization, code sharing, and runtime generic dictionaries.
- **Jeffrey Richter, *CLR via C#: The Definitive Guide to the Common Language Runtime.*** Used to describe CLR internal execution mechanisms, generic instantiation, and the difference between value-type specialization and reference-type sharing.
- **James Gosling et al., *The Java Language Specification.*** Used for the comparison with Java generics, especially type erasure and the loss of generic runtime information.
- **Tim Lindholm et al., *The Java Virtual Machine Specification.*** Used for the analysis of JVM runtime representation and bytecode execution model.
- **David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide.*** Used for the comparison with C++ templates, compile-time instantiation, and monomorphization.

2.9 prompt 6: Reflection of C++ templates

goal: Generate a detailed academic section of reflection and runtime type information of C++ templates, with explicit example. **prompt**:

```
System Persona

Act as an expert computer scientist and technical-academic writer
specializing in programming language theory, compiler design, and
```

C++ software engineering.

Goal

Generate a rigorous, technical-academic report section written in compilation-ready LaTeX that analyzes Reflection and Runtime Type Information (RTTI) specifically within the context of C++ templates, utilizing the provided theoretical framework regarding generic type representation.

Requirements

* Use the following theoretical text as the foundation for the analysis: "The availability of generic type information at runtime depends directly on the chosen representation strategy. When type erasure is employed, generic parameters are generally unavailable after compilation. Reflection mechanisms therefore provide limited access to instantiated generic types because most type information has been discarded. Conversely, implementations based on reified generics preserve concrete type information throughout compilation. \emph{Runtime reflection or runtime type identification (RTTI)} may therefore recover the actual instantiated types, enabling operations such as generic type inspection, specialized serialization, and runtime metadata analysis."

* Analyze how C++ templates align with the concepts of type preservation and reification, detailing how C++ generates specific instantiations (monomorphization) that preserve concrete type information.

* Detail the application and behavior of standard C++ RTTI mechanisms (e.g., ‘typeid‘, ‘std::type_info‘, ‘dynamic_cast‘) when applied to instantiated template classes and functions.

* Distinguish clearly between compile-time type information (available during template instantiation and metaprogramming) and runtime reflection mechanisms.

* ENFORCE STRICT SCOPE CONTROL: Confine the generated text strictly to the topic of Reflection and RTTI in C++ templates. Actively avoid tangential subjects such as memory footprint, hardware performance, code bloat, garbage collection overhead, or broader language comparisons unless explicitly demanded to illustrate the C++ mechanism.

* Provide a logical progression from the theoretical foundations of type representation strategies to the specific technical mechanisms implemented in the C++ compiler.

```
Writing Style Constraint

* Produce a complete, academically rigorous technical report section
rather than a short summary.

* Maintain a formal, objective, and scientific writing style.

* Use precise terminology appropriate for computer science literature
and official C++ standard specifications.

* Avoid unsupported claims and clearly identify technical assumptions.

* Do not use first-person forms ("I", "we").

Output Structure

* The output must consist entirely of perfectly formatted,
compilation-ready LaTeX code.

* Begin the output with an appropriate section heading
(e.g., \section{Reflection and Runtime Type Information in C++ Templates}).

* Include appropriate LaTeX structural elements, such as subsections
(\subsection), text formatting commands (\emph, \texttt), and paragraphs
to organize the logical flow of the technical analysis.

* Do not output a full document preamble (e.g., \documentclass or
\begin{document}); provide only the self-contained section content
ready to be included in a larger academic LaTeX document.
```

Primary resources

- **Bjarne Stroustrup, *The C++ Programming Language*, 4th Edition, Addison-Wesley Professional, 2013.** This reference provides the general description of C++ templates, template instantiation, and the standard RTTI mechanisms used in the analysis [**?**].
- **David Vandevoorde, Nicolai M. Josuttis, and Douglas Gregor, *C++ Templates: The Complete Guide*, 2nd Edition, Addison-Wesley Professional, 2017.** This reference provides the theoretical and technical foundation for template instantiation, specialization, and compile-time preservation of template type information [**?**].
- **cppreference.com, *C++ Language Reference: RTTI and typeid*, 2025.** This reference was used for the description of the standard RTTI facilities, including `typeid`, `std::type_info`, and `dynamic_cast` behavior [**?**].